Coverage-Based Testing: Measuring and Interpreting Coverage with `coverage.py` and `pytest-cov`

SE 413/513 — Software Testing

Course Reading Material

Unit: White-Box Testing and Coverage

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,
author = {Hasan, Kamrul},
year = {2026},
title = {Software Testing},
institution = {Grand Valley State University},
url = {https://github.com/mdkamrulhasan/software-testing}
}
```

Contents

1 Learning Objectives **3**

2 Recap: From Coverage Criteria to Coverage Tools **3**

3 What Is Coverage-Based Testing? **3**

4 Types of Code Coverage **4**
4.1 Statement (Line) Coverage 4
4.2 Branch (Decision) Coverage 4
4.3 Function Coverage 4
4.4 Other Criteria at a Glance 5

5 The Running Example **5**

6 Measuring Coverage with `coverage.py` **6**
6.1 Installing and Running 6
6.2 The Basic Report 7
6.3 Showing What Was Missed: `-m` 7
6.4 Turning On Branch Coverage 7
6.5 Other Useful Commands 8

6.6 Making Settings Permanent 8

7 Integrating Coverage with `pytest`: `pytest-cov` 8
7.1 Enforcing a Minimum 9

8 Interpreting Coverage Reports 10
8.1 Reading the Terminal Table 10
8.2 Reading the `Missing` Column 10
8.3 Reading the HTML Report 11
8.4 Machine-Readable Reports 11

9 Function Coverage in Practice 12

10 From Report to Tests: Closing the Gaps 12

11 What Coverage Cannot Tell You 14
11.1 Limitation 1: Execution Is Not Verification 14
11.2 Limitation 2: Branches Are Not Conditions 15
11.3 Limitation 3: Coverage Cannot See Boundaries 15
11.4 Limitation 4: Coverage Says Nothing About Missing Code 15
11.5 What the Research Says 15

12 Best Practices 16

13 Common Mistakes 16

14 Applying This to Your Semester Project 17

15 A Practical Mental Model 17

16 Summary 18

17 Practice Exercises 18

18 Discussion Questions 19

19 Key Takeaways for SE 413/513 20

20 References 20

1 Learning Objectives

After completing this reading, you should be able to:

- Explain what coverage-based testing is, and why coverage is used as a *test adequacy* measure rather than a correctness measure.
- Define statement, branch, and function coverage, and compute each for a small Python module.
- Use `coverage.py` to run tests under measurement and produce terminal, HTML, JSON, and XML reports.
- Use `pytest-cov` to collect coverage as part of an ordinary `pytest` run, including enforcing a minimum with `--cov-fail-under`.
- Read every column of a coverage report, including the `Missing` column's line ranges and partial-branch arrows (`16->18`).
- Use a coverage report to locate untested or inadequately tested code, and turn each gap into a meaningful new test.
- Explain the limitations of coverage metrics, and apply best practices for using them on a real project.

2 Recap: From Coverage Criteria to Coverage Tools

The Week 5 reading on control-flow graphs defined **statement**, **branch**, and **path coverage**, placed them in a subsumption hierarchy (path ⇒ branch ⇒ statement), and briefly showed one `pytest-cov` command. The Week 6 reading added **data-flow** criteria (All-Defs, All-Uses, All-DU-Paths), and noted that mainstream tools measure statement and branch coverage, not DU pairs.

Key Idea

This reading assumes that vocabulary. If "branch coverage subsumes statement coverage" or "a CFG edge" are unfamiliar, revisit the Week 5 reading first — we will not re-derive the criteria here. This week is about the *practice*: measuring coverage with real tools, reading the reports accurately, and deciding what to do about what they show.

3 What Is Coverage-Based Testing?

Coverage-based testing uses a measure of *how much of the program's structure a test suite exercises* to guide and evaluate testing[4, 5]. Its purpose is not to say whether the program is correct — only the assertions in your tests can do that — but to answer a narrower and very practical question:

"Which parts of the code did my tests never even run?"

Code that no test executes is code whose behavior is completely unchecked. Coverage-based testing makes that code visible, so the tester can decide whether it deserves a test, is dead and should be removed, or is deliberately left untested.

In the terminology of the testing literature, coverage is a **test adequacy criterion**: a rule for judging when a test suite has done "enough" of a particular kind of exercise[4]. Coverage-based testing typically serves three purposes:

1. **Finding gaps.** A report highlights lines and branches no test reached — often error handling, rarely used options, or newly added code.
2. **Guiding new tests.** Each gap points to an input or scenario the suite has not yet considered.
3. **Guarding against regression in the suite itself.** Tracked over time (for example, in continuous integration), coverage reveals when new code is merged without tests.

Key Idea

Coverage measures **execution**, not **verification**. A line counts as "covered" the moment it runs, whether or not any assertion ever checks what it did. Low coverage is reliable bad news; high coverage is, at best, weak good news (Section 11).

4 Types of Code Coverage

Many coverage criteria exist. This course emphasizes the three that Python's standard tooling can report directly.

4.1 Statement (Line) Coverage

$$\text{Statement coverage} = \frac{\text{executable statements executed by at least one test}}{\text{total executable statements}} \times 100\%$$

The simplest and most widely reported criterion. `coverage.py` measures it by default. In Python, "statement" is close to, but not exactly, "line": a multi-line statement counts once, and blank lines and comments are not counted at all.

4.2 Branch (Decision) Coverage

$$\text{Branch coverage} = \frac{\text{branch outcomes taken by at least one test}}{\text{total branch outcomes}} \times 100\%$$

Every `if`, `elif`, `while`, and `for` creates two possible outcomes (for loops: "enter the body" and "exit the loop"). Branch coverage requires each outcome to occur at least once. It subsumes statement coverage, and it catches the classic gap statement coverage hides: an `if` with no `else` whose *false* outcome is never taken. In `coverage.py`, it must be switched on explicitly (`--branch`).

4.3 Function Coverage

$$\text{Function coverage} = \frac{\text{functions called at least once}}{\text{total functions}} \times 100\%$$

The coarsest criterion: it only asks whether each function was entered at all. It is weak on its own — a function can be "covered" by a single call that checks nothing — but it is an excellent *first* filter: a function with 0% coverage is a whole unit of behavior with no test, and it is usually the first gap worth closing.

4.4 Other Criteria at a Glance

Criterion	Requires that tests...	In `coverage.py`?
Function	call every function at least once	Per-function table in HTML report
Statement	execute every statement	Yes (default)
Branch	take every decision both ways	Yes (`--branch`)
Condition	make every boolean sub-condition both true and false	No
MC/DC	show each condition independently affects the decision	No
Path	execute every entry-to-exit path	No (often infeasible; Week 5)
Data-flow	exercise def-use pairs	No (Week 6)

Condition coverage and MC/DC (modified condition/decision coverage) matter in safety-critical domains such as avionics[5]; we mention them so that the gap they fill is clear when we reach the limitations in Section 11.

5 The Running Example

All output in this reading comes from real runs of `coverage.py` 7.16 and `pytest-cov` on the following small module. Line numbers are shown because coverage reports refer to them.

```
"""A small checkout module used in the Week 7 coverage examples."""


def subtotal(items):
    total = 0.0
    for price, qty in items:
        if qty < 0:
            raise ValueError("quantity cannot be negative")
        total += price * qty
    return total


def apply_coupon(amount, code):
    if code == "SAVE10":
        amount = amount * 0.90
    elif code == "FLAT5":
        amount = max(amount - 5, 0)
    return round(amount, 2)


def shipping_fee(amount, is_member):
    if is_member or amount >= 50:
        return 0.0
    return 7.99


def format_receipt(amount):
    return f"Total due: ${amount:.2f}"
```

The module lives at `checkout_app/checkout.py`. A first, deliberately thin test suite, `tests/test_checkout_v1.py`, looks like many real first attempts — one happy-path test per function, and one function forgotten entirely:

```
from checkout_app.checkout import subtotal, apply_coupon, shipping_fee


def test_subtotal_two_items():
    assert subtotal([(10.0, 2), (5.0, 1)]) == 25.0


def test_save10_coupon():
    assert apply_coupon(100.0, "SAVE10") == 90.0


def test_member_ships_free():
    assert shipping_fee(20.0, True) == 0.0
```

All three tests pass. The question for the rest of this reading is: *how much of* ***`checkout.py`*** *have they actually exercised?*

6 Measuring Coverage with `coverage.py`

`coverage.py` is the standard coverage tool for Python[2]. It works by watching the interpreter as your tests run, recording which lines (and, optionally, which line-to-line transitions) executed, and saving that data for later reporting.



The key design point is the separation between *measuring* (`coverage run`) and *reporting* (`coverage report`, `html`, ...). You measure once and can produce as many report formats as you like from the same data.

6.1 Installing and Running

```
pip install coverage pytest

# Run pytest under coverage measurement
coverage run -m pytest tests/test_checkout_v1.py

# Summarize the results
coverage report
```

Tip

`coverage run -m pytest` is to `pytest` what `python -m pytest` is to `pytest`: the same test run, but launched through a wrapper. Anything you can run with `python`, you can run with `coverage run` — including a plain script, which is useful for measuring an exploratory or manual run.

6.2 The Basic Report

```
Name                          Stmts   Miss  Cover
-----------------------------------------------
checkout_app/__init__.py          0      0   100%
checkout_app/checkout.py         19      5    74%
tests/__init__.py                 0      0   100%
tests/test_checkout_v1.py         7      0   100%
-----------------------------------------------
TOTAL                            26      5    81%
```

Two things stand out. First, `checkout.py` is at 74% statement coverage: 14 of 19 statements ran. Second, the *test files themselves* are included in the total, inflating it to 81%. Test code almost always reaches 100% (every test runs), so including it makes a suite look better than it is. Restrict measurement to your application code with `--source`:

```
coverage run --source=checkout_app -m pytest tests/test_checkout_v1.py
```

6.3 Showing What Was Missed: `-m`

The percentage alone is not actionable. `coverage report -m` ("show missing") adds the line numbers that never ran:

```
Name                          Stmts   Miss  Cover   Missing
-----------------------------------------------------------
checkout_app/checkout.py         19      5    74%   8, 16-17, 24, 28
```

Mapping these back to the listing in Section 5: line 8 is the negative-quantity `raise`; lines 16–17 are the `FLAT5` coupon; line 24 is the paid-shipping `return 7.99`; line 28 is the body of `format_receipt()`.

6.4 Turning On Branch Coverage

```
coverage run --branch --source=checkout_app -m pytest tests/test_checkout_v1.py
coverage report -m
```

```
Name                          Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------
checkout_app/checkout.py         19      5     10      3    66%   8, 16-17, 24, 28
```

The percentage *dropped* from 74% to 66%, even though the tests did not change. That is not a malfunction: the report is now counting something it previously ignored. Section 8 explains each new column and exactly how 66% is computed.

Common Pitfall

By default, `coverage.py` reports **statement coverage only**. A number quoted without saying whether branch measurement was on is ambiguous — and the statement-only number is always the more flattering one. Make `--branch` your default (Section 6.6 shows how to make it permanent).

6.5 Other Useful Commands

Command	Purpose
`coverage run -m pytest`	Run tests and record coverage data in `.coverage`
`coverage report -m`	Terminal summary with missing lines
`coverage report --skip-covered`	Hide files already at 100% (useful on large projects)
`coverage html`	Browsable report in `htmlcov/index.html`
`coverage json` / `coverage xml`	Machine-readable reports for scripts and CI services
`coverage erase`	Delete previously collected data
`coverage combine`	Merge data files from several runs (e.g., parallel test processes)

6.6 Making Settings Permanent

Rather than retyping flags, put them in the project's `pyproject.toml` (a `.coveragerc` file works too)[2]:

```
[tool.coverage.run]
branch = true
source = ["checkout_app"]

[tool.coverage.report]
show_missing = true
fail_under = 90
exclude_also = [
    "if __name__ == .__main__.:",
]
```

With this file in place, a plain `coverage run -m pytest` followed by `coverage report` measures branches, ignores test files, lists missing lines — and **fails** if total coverage is below 90%:

```
Coverage failure: total of 66 is less than fail-under=90
```

(the command exits with a nonzero status, which is what lets a CI pipeline stop on it). `exclude_also` lists regular expressions for lines that should not count; we return to exclusions, and their risks, in Section 12.

7 Integrating Coverage with `pytest`: `pytest-cov`

`pytest-cov` is a `pytest` plugin that drives `coverage.py` for you[3]. Instead of the two-step `coverage run` / `coverage report`, coverage becomes a set of options on your normal test command:

```
pip install pytest-cov
pytest --cov=checkout_app --cov-branch --cov-report=term-missing
```

```
================================ tests coverage ================================
Name                          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
checkout_app/__init__.py          0      0      0      0   100%
checkout_app/checkout.py         19      5     10      3    66%   8, 16-17, 24, 28
-------------------------------------------------------------------------
TOTAL                            19      5     10      3    66%
3 passed in 0.03s
```

The numbers are identical to the `coverage.py` run — `pytest-cov` uses the same engine. Its advantages are convenience, a single command for CI, and correct handling of features such as parallel test execution with `pytest-xdist`.

Option	Effect
`--cov=PKG`	Measure coverage of package/directory `PKG` (bare `--cov` uses the config file's `source`)
`--cov-branch`	Enable branch measurement
`--cov-report=term-missing`	Terminal table with the `Missing` column
`--cov-report=html`	Write `htmlcov/`
`--cov-report=xml` / `json`	Write `coverage.xml` / `coverage.json`
`--cov-fail-under=N`	Make the test run fail if total coverage $< N\%$
`--cov-context=test`	Record *which test* executed each line (viewable in the HTML report)

Several `--cov-report` options can be combined in one run:

```
pytest --cov --cov-report=term-missing --cov-report=html --cov-report=xml
```

Tip

`pytest-cov` reads the same `[tool.coverage.*]` settings as `coverage.py`. Put the configuration in `pyproject.toml` once; then `pytest --cov` and `coverage run -m pytest` will agree with each other, and with your teammates' and your CI's runs.

7.1 Enforcing a Minimum

```
pytest --cov=checkout_app --cov-branch --cov-fail-under=80
```

```
FAIL Required test coverage of 80% not reached. Total coverage: 65.52%
3 passed in 0.03s
```

Notice that all three *tests passed*, yet the *command* exits with status 1. In a CI pipeline, that nonzero exit marks the build as failed. A minimal GitHub Actions step looks like this:

```
- name: Run tests with coverage
  run: |
    pip install pytest pytest-cov
    pytest --cov --cov-branch --cov-report=xml --cov-fail-under=80
```

We will return to coverage in CI pipelines in the Continuous Testing week.

8 Interpreting Coverage Reports

8.1 Reading the Terminal Table

Column	Meaning
`Stmts`	Executable statements in the file
`Miss`	Statements never executed
`Branch`	Possible branch outcomes (two per decision, when `--branch` is on)
`BrPart`	**Partial** branches: decisions that ran, but took only *one* of their outcomes
`Cover`	Combined coverage percentage (formula below)
`Missing`	Line numbers never executed, and partial-branch arcs (`a->b`)

How the percentage is computed. With branch measurement on, `coverage.py` combines statements and branch outcomes into a single number[2]:

$$\text{Cover} = \frac{\text{executed statements} + \text{executed branch outcomes}}{\text{statements} + \text{branch outcomes}}$$

For our v1 suite: 14 of 19 statements ran, and 5 of 10 branch outcomes were taken, so

$$\text{Cover} = \frac{14 + 5}{19 + 10} = \frac{19}{29} \approx 65.5\% \;\rightarrow\; 66\%.$$

Separately, statement coverage is $14/19 \approx 74\%$ and branch coverage is $5/10 = 50\%$. The HTML report shows all three numbers side by side (Figure 1).

Where the 10 branch outcomes come from. There are five decisions in `checkout.py`, each with two outcomes: the `for` loop on line 6 (enter body / exit loop), and the conditions on lines 7, 14, 16, and 22.

Where the 3 partial branches come from.

- Line 7, `if qty < 0`: was always *false* — no test passed a negative quantity.
- Line 14, `if code == "SAVE10"`: was always *true* — the only coupon test used `SAVE10`.
- Line 22, `if is_member or amount >= 50`: was always *true* — the only test used a member.

Line 16 (`elif code == "FLAT5"`) is not *partial*; it is *missing* entirely, since it never ran at all. Both of its outcomes count among the missed branches.

8.2 Reading the `Missing` Column

The `Missing` column uses three notations:

Notation	Meaning
`8`	Line 8 never executed
`16-17`	Lines 16 through 17 never executed
`16->18`	Line 16 ran, but control never went *directly* from line 16 to line 18 — a branch outcome was never taken

The v1 output contains no arrows, even though it has three partial branches. That is because each missed arc in v1 ends at a line that is already listed as missing (for example, $7 \rightarrow 8$ is implied

by "8"), so `coverage.py` does not repeat it. An arrow appears when the missed arc's destination *did* run by some other route. For instance, a suite that tests only the `SAVE10` and `FLAT5` coupons produces:

```
checkout_app/checkout.py       19      10      10       1    41%   5-10, 16->18, 22-24, 28
```

Line 18 (`return round(amount, 2)`) ran — both coupon tests reach it — but never *directly* from line 16. The `16->18` arc is the "neither coupon matched" outcome: **no test used an unrecognized coupon code**.

Key Idea

Every arrow `a->b` in a report translates into a concrete question: *"What input makes the decision on line a go the way that leads straight to line b?"* The answer is your next test.

8.3 Reading the HTML Report

The terminal table is enough for small modules, but the HTML report (`coverage html`, or `--cov-report=html`) is far easier to work with on real code. Open `htmlcov/index.html` in a browser.

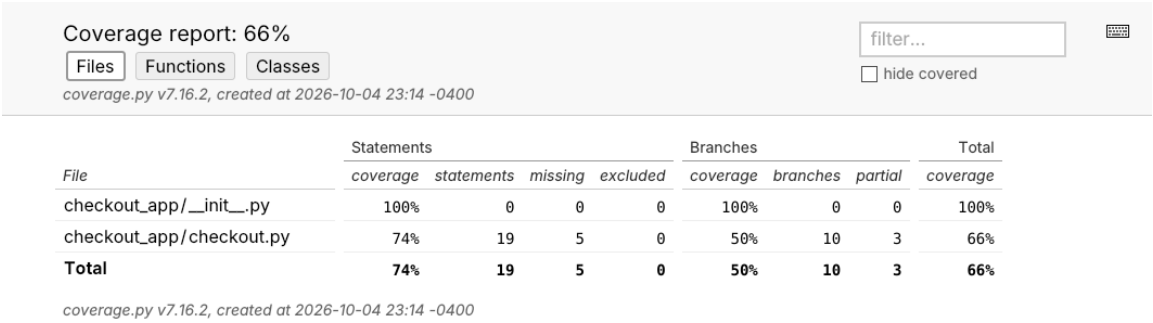


Figure 1: The HTML report's index page for the v1 suite. Statement coverage (74%), branch coverage (50%), and the combined total (66%) are reported separately. The *Functions* and *Classes* buttons switch to per-function and per-class views.

Clicking a file opens an annotated copy of its source (Figure 2):

Tip

The buttons at the top of the file view (*run*, *missing*, *excluded*, *partial*) toggle highlighting for each category. On a large file, turning off *run* and leaving only *missing* and *partial* on gives you a to-do list of gaps.

8.4 Machine-Readable Reports

`coverage json` and `coverage xml` contain the same information in a form other programs can consume. The XML output follows the Cobertura format, which most CI services and code-review tools understand for displaying coverage on pull requests. The JSON report is convenient for your own scripts — for example, its per-function section for the v1 suite lists `format_receipt` at 0%, which brings us to function coverage.



Figure 2: Annotated source view of `checkout.py` under the v1 suite. **Green** bar: executed. **Red** background: never executed. **Yellow** background: partial branch, with the missed arc shown at the right (e.g., 7 ↛ 8). Hovering over a yellow line shows an explanation such as "line 7 didn't jump to line 8 because the condition on line 7 was never true."

9 Function Coverage in Practice

`coverage.py` does not print a single "function coverage" percentage, but recent versions (7.5 and later) add a **Functions** index to the HTML report that breaks coverage down per function (Figure 3).

Reading this view as function coverage: `subtotal`, `apply_coupon`, and `shipping_fee` were each entered at least once; `format_receipt` shows 0%. Function coverage is therefore $3/4 = 75\%$.

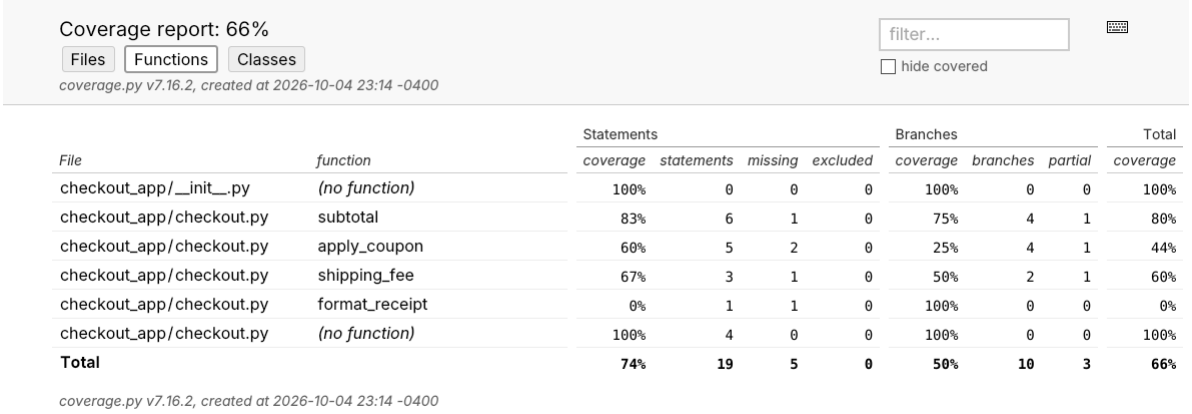
The per-function view is also a quick way to rank where testing effort is most needed: `apply_coupon` (44% total) is the least-tested function that *has* tests, and `format_receipt` has none at all.

Common Pitfall

A function can be at "100% function coverage" and still have most of its body unexercised — `apply_coupon` was "covered" at the function level by a single `SAVE10` test. Treat function coverage as a coarse first filter, then use statement and branch coverage to look inside each function.

10 From Report to Tests: Closing the Gaps

A coverage report is only useful if it changes what you test. The disciplined way to use it is to treat each gap as a question about *behavior*, not about a line number:



Coverage report: 66%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

filter...

hide covered

		Statements				Branches			Total
File	function	coverage	statements	missing	excluded	coverage	branches	partial	coverage
checkout_app/__init__.py	*(no function)*	100%	0	0	0	100%	0	0	100%
checkout_app/checkout.py	subtotal	83%	6	1	0	75%	4	1	80%
checkout_app/checkout.py	apply_coupon	60%	5	2	0	25%	4	1	44%
checkout_app/checkout.py	shipping_fee	67%	3	1	0	50%	2	1	60%
checkout_app/checkout.py	format_receipt	0%	1	1	0	100%	0	0	0%
checkout_app/checkout.py	*(no function)*	100%	4	0	0	100%	0	0	100%
Total		**74%**	**19**	**5**	**0**	**50%**	**10**	**3**	**66%**

coverage.py v7.16.2, created at 2026-10-04 23:14 -0400

Figure 3: The HTML report's Functions view for the v1 suite. The "(no function)" rows are module-level code: imports and `def` statements themselves, which run when the module is imported.

Gap	Behavior never checked	New test
line 8 / 7 ↛ 8	Negative quantity is rejected	`subtotal([(10.0, -1)])` raises `ValueError`
lines 16–17	`FLAT5` subtracts $5	`apply_coupon(100.0, "FLAT5") == 95.0`
(line 17, `max`)	`FLAT5` never goes negative	`apply_coupon(3.0, "FLAT5") == 0.0`
16 → 18	Unknown code leaves amount unchanged	`apply_coupon(100.0, "BOGUS") == 100.0`
line 24 / 22 ↛ 24	Non-members with small orders pay shipping	`shipping_fee(20.0, False) == 7.99`
line 28	Receipt formatting	`format_receipt(12.5) == "Total due:  $12.50"`

Note the third row: the `max(amount - 5, 0)` floor is *one line*, so a single `FLAT5` test covers it — but the interesting behavior (never charging a negative total) only happens when the amount is below $5. Coverage will never ask for that test; your understanding of the specification must. The improved suite (`test_checkout_v2.py`):

```
import pytest
from checkout_app.checkout import (
    subtotal, apply_coupon, shipping_fee, format_receipt,
)


def test_subtotal_two_items():
    assert subtotal([(10.0, 2), (5.0, 1)]) == 25.0


def test_subtotal_rejects_negative_quantity():
    with pytest.raises(ValueError):
        subtotal([(10.0, -1)])


@pytest.mark.parametrize("amount, code, expected", [
```

```
    (100.0, "SAVE10", 90.0),
    (100.0, "FLAT5", 95.0),
    (3.0, "FLAT5", 0.0),
    (100.0, "BOGUS", 100.0),
])
def test_apply_coupon(amount, code, expected):
    assert apply_coupon(amount, code) == expected


def test_member_ships_free():
    assert shipping_fee(20.0, True) == 0.0


def test_non_member_small_order_pays_shipping():
    assert shipping_fee(20.0, False) == 7.99


def test_format_receipt():
    assert format_receipt(12.5) == "Total due: $12.50"
```

```
Name                        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------
checkout_app/checkout.py       19      0     10      0   100%
-----------------------------------------------------------------------
TOTAL                          19      0     10      0   100%
9 passed in 0.04s
```

100% statement, 100% branch, 100% function coverage. Is `checkout.py` now fully tested? The next section shows why the answer is *no*.

11 What Coverage Cannot Tell You

11.1 Limitation 1: Execution Is Not Verification

The following test also produces **100% statement and branch coverage** of `checkout.py` — verified with the same command:

```
def test_runs_everything():
    for code in ["SAVE10", "FLAT5", "NONE"]:
        apply_coupon(100.0, code)
    shipping_fee(20.0, True)
    shipping_fee(20.0, False)
    format_receipt(1)
    subtotal([(1.0, 1)])
    try:
        subtotal([(1.0, -1)])
    except ValueError:
        pass
```

It contains **no assertions**. It would pass if `apply_coupon` returned `None`, if `FLAT5` added $5 instead of subtracting it, or if every shipping fee were zero. The only defects it can detect are ones that raise an unexpected exception. Coverage tools cannot see assertions; they only see execution[8].

11.2 Limitation 2: Branches Are Not Conditions

In `shipping_fee()`, line 22 is `if is_member or amount >= 50`. The v2 suite takes this decision both ways — true via `is_member=True`, false via a non-member $20 order — so branch coverage is satisfied. But because `or` short-circuits, the sub-condition `amount >= 50` has **never once evaluated to true**. The "free shipping over $50" rule is completely unverified, yet the report shows no gap. This is precisely the gap *condition coverage* and *MC/DC* (Section 4) exist to close — and `coverage.py` does not measure them.

11.3 Limitation 3: Coverage Cannot See Boundaries

Suppose a developer accidentally changes line 22 to `amount > 50`, so that a $50.00 order is now charged shipping. We ran the v2 suite against this modified code:

```
TOTAL                           19      0     10      0   100%
9 passed in 0.03s
```

Every test passes, and coverage is still 100%. Only a test at the boundary — `shipping_fee(50.0, False) == 0.0`, exactly what boundary-value analysis from Week 3 prescribes — would detect it. This is the same lesson as the Week 5 shipping-weight example: coverage tells you a branch was taken, not whether it was taken with the inputs most likely to expose a defect.

Tip

Deliberately planting a small bug (like `>=` → `>`) and checking whether any test fails is the core idea of **mutation testing**. Tools such as `mutmut` automate it, producing a "mutation score" that measures how well a suite *detects* faults, not merely how much code it runs. It is a natural complement to coverage, and an advanced topic later in the course.

11.4 Limitation 4: Coverage Says Nothing About Missing Code

A coverage tool can only measure code that exists. If the specification requires that coupon codes be case-insensitive (`"save10"` should work) and the implementation simply never handles it, there is no line to be uncovered. Only specification-based (black-box) testing can reveal missing functionality.

11.5 What the Research Says

Empirical studies support caution. Inozemtseva and Holmes measured many test suites for large Java programs and found that, once suite size is controlled for, coverage is only weakly to moderately correlated with a suite's ability to detect faults[9]. Coverage is a good indicator of what is *untested*, but a poor target to optimize on its own.

Real-World Case Study

Code coverage at Google. Google has reported on coverage use across its codebase, measured for over a billion lines of code and used daily in code review[10]. Developers surveyed largely found coverage useful — particularly the coverage of the *changed lines* shown during code review, which flagged new code arriving without tests. Rather than a single mandated threshold, Google's published guidance offers rough bands (around 60% as acceptable, 75% as commendable, and 90% as exemplary) and explicitly cautions against treating any number as a goal in itself, emphasizing that coverage of newly changed code matters more than the project-wide total[11].

12 Best Practices

1. **Measure branches, not just statements.** Turn on branch measurement permanently in `pyproject.toml`.
2. **Measure the right code.** Use `source` (or `--cov=PKG`) so test files and third-party code do not inflate the total.
3. **Read the gaps, not the percentage.** The `Missing` column and the HTML report are the useful parts of the output.
4. **Turn each gap into a behavior-level test with real assertions** (Section 10) — never a test that merely executes the line.
5. **Combine coverage with black-box design.** Use equivalence partitioning and boundary-value analysis to choose *values*; use coverage to confirm nothing in the implementation was left untouched.
6. **Set a floor, not a target.** `--cov-fail-under` is a useful guard against regressions, but set it at a level the team agrees on, and raise it gradually. Pay special attention to coverage of newly changed code.
7. **Exclude sparingly and visibly.** Mark genuinely untestable lines with `# pragma: no cover` or `exclude_also`, for example a `if __name__ == "__main__":` launcher or a defensive branch for a platform you do not run on. Every exclusion should be explainable in code review; excluding code because it is *hard* to test hides exactly the gaps coverage exists to show.
8. **Investigate 0% functions and long red blocks first.** They are entire behaviors with no test, and usually the cheapest gaps to close.

13 Common Mistakes

Common Pitfall

Quoting a statement-only number as "coverage." Without `--branch`, the same suite looked 8 percentage points better (74% vs. 66%). Always state which criterion a number measures.

Common Pitfall

Including test files in the measurement. Test code is nearly always 100% executed; counting it inflates the total.

Common Pitfall

Writing tests to hit lines. Assertion-free or weak-assertion tests raise coverage without improving fault detection (Limitation 1). This is Goodhart's Law in action: once the number becomes the target, it stops measuring what you cared about.

Common Pitfall

Treating 100% as "done." The v2 suite was at 100% and still missed the $50 free-shipping rule and an off-by-one at the boundary (Limitations 2–3).

Common Pitfall

Forgetting stale data. `coverage run` replaces the previous `.coverage` data file by default, but tools that append or combine data can mix old runs into a new report. When numbers look surprising, `coverage erase` and run again.

14 Applying This to Your Semester Project

Coverage reports are a natural form of *preliminary results* for the test-implementation milestone of the semester project. A useful way to present coverage in a progress report:

- State the exact command and configuration (statement or branch? which packages measured?).
- Report coverage *per module*, not only the total, and include the per-function view for the modules you are focusing on.
- For the most important remaining gaps, say what behavior is untested and whether you plan to test it, exclude it (with a reason), or remove it as dead code.
- Pair the coverage numbers with at least one example of a test designed from the specification (boundary values, equivalence classes) — evidence that your suite does more than execute lines.

For Track A projects, running coverage on the existing open-source test suite *before* adding your own tests gives you a baseline that makes your contribution measurable.

15 A Practical Mental Model

When you see a coverage percentage, think:

> "Which criterion, over which code — and where are the gaps?"

When you see a red line in the HTML report, think:

> "What behavior does this line implement, and what input would make it run?"

When you see `a->b` in the `Missing` column, think:

> "The decision on line `a` was never decided in the direction that leads to `b`. What input decides it that way?"

When you see 100%, think:

> "Every line ran. Did my assertions check the results, did I test the boundaries, and did every sub-condition matter?"

16 Summary

Command / Option	Purpose
`coverage run -m pytest`	Run tests under measurement
`coverage run --branch`	Also measure branch outcomes
`coverage report -m`	Terminal table with missing lines/arcs
`coverage html`	Annotated, browsable report in `htmlcov/`
`coverage json` / `xml`	Machine-readable reports (scripts, CI)
`pytest --cov=PKG --cov-branch`	Same measurement, via the `pytest-cov` plugin
`--cov-report=term-missing`	Terminal table with `Missing` column
`--cov-fail-under=N`	Fail the run below `N`% total coverage
`[tool.coverage.*]` in `pyproject.toml`	Shared, permanent configuration
`# pragma: no cover`	Exclude a line/block from measurement (use sparingly)

Criterion	Catches	Misses
Function	Whole functions with no test	Almost everything inside a function
Statement	Lines never executed (dead or unreached code)	Untaken `false` outcomes of an `if` with no `else`
Branch	Decision outcomes never taken	Short-circuited sub-conditions, boundary values, weak assertions, missing code

Key Idea

Coverage finds what you did not test; it cannot certify what you did. Use it as a flashlight for gaps, then fill those gaps with tests designed from the specification and backed by real assertions.

17 Practice Exercises

Exercise 1 — Computing the Numbers

A module has 40 statements and 12 branch outcomes. A test suite executes 34 statements and 8 branch outcomes. Compute (a) statement coverage, (b) branch coverage, and (c) the combined `Cover` percentage that `coverage.py` would report with branch measurement on.

Exercise 2 — Reading a Report

A different suite for the running example calls `subtotal([])`, `apply_coupon(100.0, "FLAT5")`, `shipping_fee(80.0, False)`, and `format_receipt(1)`, and produces:

```
Name                         Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------
checkout_app/checkout.py        19      5     10      4    62%   7-9, 15, 16->18, 24
```

(a) Using the listing in Section 5, explain in words what each entry in the `Missing` column means. (b) Identify the four partial branches. (c) Verify the 62% figure using the formula in Section 8. (d) Write one test that removes each `Missing` entry. (Hint for `7-9`: why did the loop body never run?)

Exercise 3 — Hands-On

Clone the course repository, add `checkout.py` and `test_checkout_v1.py` to a scratch project, and (a) reproduce the 74% and 66% numbers; (b) generate the HTML report and find the three yellow lines; (c) add tests one at a time, re-running coverage after each, until you reach 100% branch coverage.

Exercise 4 — Beyond 100%

Starting from the v2 suite (100% branch coverage), add the tests needed to (a) make `amount >= 50` evaluate to true at least once, and (b) detect the `>= 50` → `> 50` change from Limitation 3. Verify (b) by making the change and confirming that a test fails.

Exercise 5 — Design Question

Your team lead proposes adding `--cov-fail-under=100` to CI "so nobody can merge untested code." Using at least two concepts from this reading, argue for or against this policy, and propose an alternative if you argue against it.

18 Discussion Questions

1. Why is coverage described as a test *adequacy* criterion rather than a *correctness* criterion?
2. Why did turning on `--branch` *lower* the reported percentage for the same test suite?
3. What is a partial branch, and why does line 16 in the running example count as missing rather than partial under the v1 suite?
4. Why does the `Missing` column sometimes show arrows and sometimes not, even when partial branches exist?
5. Function coverage is easy to achieve. Why is it still worth looking at?
6. How can a test suite reach 100% branch coverage while a sub-condition of a decision is never true? Which coverage criterion would catch this?
7. Why can coverage never reveal a missing feature?
8. What are the risks of setting a high, project-wide coverage threshold? What does focusing on coverage of *changed* code offer instead?

9. When is `# pragma: no cover` justified, and when is it a warning sign?
10. How do coverage-based (white-box) and specification-based (black-box) testing complement each other? Use an example from this reading.

19 Key Takeaways for SE 413/513

1. **Coverage-based testing measures how much of the code's structure the tests execute**, to find and close gaps — not to prove correctness.
2. **Statement, branch, and function coverage are the practical criteria in Python;** branch is the one to report by default.
3. **`coverage.py` measures, then reports;** `pytest-cov` does both inside your normal `pytest` run.
4. **The `Missing` column and the HTML report are where the value is:** every red line and every `a->b` arc is a question about untested behavior.
5. **Close gaps with behavior-level tests and real assertions,** chosen with black-box techniques where values matter.
6. **100% coverage is not "done":** assertions, sub-conditions, boundaries, and missing features are all invisible to a coverage tool.

20 References

- `coverage.py`: measurement, branch coverage, configuration, exclusions, and report formats.[2]
- `pytest-cov`: integrating coverage with `pytest`, report options, and `--cov-fail-under`.[3]
- Coverage criteria as test adequacy criteria, and the broader family of structural criteria.[4, 5, 6]
- Coverage-based (white-box) test techniques in the ISTQB Foundation syllabus.[7]
- The limits of coverage as a quality measure.[8, 9]
- Coverage practice at scale, and guidance on thresholds.[10, 11]
- Using `pytest`, this course's default testing framework.[12]
- Course material on software testing principles and related testing practices.[1]

References

[1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. https://github.com/mdkamrulhasan/software-testing

[2] Batchelder, N., et al. (2026). *Coverage.py Documentation*. https://coverage.readthedocs.io/

[3] pytest-cov contributors. (2026). *pytest-cov Documentation*. https://pytest-cov.readthedocs.io/

[4] Zhu, H., Hall, P. A. V., & May, J. H. R. (1997). Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4), 366–427. https://doi.org/10.1145/267580.267590

[5] Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.

[6] Aniche, M. (2022). *Effective Software Testing: A Developer's Guide*. Manning Publications.

[7] International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level (CTFL) Syllabus*, v4.0.1.

[8] Marick, B. (1999). How to misuse code coverage. In *Proceedings of the 16th International Conference on Testing Computer Software*.

[9] Inozemtseva, L., & Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 435–445. https://doi.org/10.1145/2568225.2568271

[10] Ivanković, M., Petrović, G., Just, R., & Fraser, G. (2019). Code coverage at Google. In *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 955–963. https://doi.org/10.1145/3338906.3340459

[11] Arguelles, C., Ivanković, M., & Bender, A. (2020). *Code Coverage Best Practices*. Google Testing Blog. https://testing.googleblog.com/2020/08/code-coverage-best-practices.html

[12] Okken, B. (2022). *Python Testing with pytest* (2nd ed.). Pragmatic Bookshelf.